Dynamic Balanced Academic Curriculum Problem

Francisco Valencia Salgado
Departamento de Informática
Universidad Técnica Federico Santa María
Valparaíso, Chile
francisco.valencias@usm.cl

***Abstract*—Abstract: The Balanced Academic Curriculum Planning (BACP) problem involves allocating modules to academic terms, whilst complying with the constraints of prerequisites, term-based workload and the number of modules that can be allocated to each term, with the aim of maintaining a balanced academic workload across the different terms. However, this problem does not take into account the possibility that a student may fail one or more of their courses; this gives rise to the Dynamic Balanced Academic Curriculum Problem (DBACP), which is the focus of this report. However, first, an analysis will be carried out of relevant approaches proposed for the BACP , examining their main components, the cases evaluated and the published results; and based on this review, a mathematical formulation for the DBACP is presented.**

***Index Terms*—Balanced Academic Curriculum Problem, DBACP, Metaheuristics, Dynamic Replanning, Academic Load Balancing**

I. Introduction

The Balanced Academic Curriculum Problem (BACP) involves assigning a set of modules to a fixed number of academic terms, whilst respecting prerequisite relationships, minimum and maximum course loads, and limits on the number of modules assigned to each term. The problem was first formulated by Castro and Manzano in 2001 [1] and, since then, has been studied as a combinatorial optimisation problem with constraints. Its main objective is to obtain a balanced curriculum, typically by minimising the maximum credit load assigned to any academic term, thereby achieving a balanced load [2].

Several approaches have been proposed to solve the BACP, including mathematical programming and metaheuristics such as Ant Colony Optimization [3] and the Firefly Algorithm [4]. However, the BACP is based on the assumption that a student will not fail any of their modules; consequently, a predefined academic pathway is established. In practice, this is not the case: students may fail modules or progress differently from what was envisaged in the original curriculum, which renders the BACP's output useless or, at best, merely a guide. The Dynamic Balanced Academic Curriculum Problem (DBACP) extends this by taking into account each student's current academic situation. Rather than generating a single, fixed curriculum, the subjects to be allocated are re-planned based on those the student has already passed, those still pending, and the prerequisite relationships that remain active [5].

The aim of this paper is to review the relevant approaches proposed for the BACP and its variants, analysing the solution techniques, their main components, the reference cases and the real-world cases used for evaluation, as well as the results reported by the authors. Furthermore, a mathematical formulation of the DBACP is presented, which incorporates the student's current academic status and the dynamic re-planning of the remaining curriculum. The remainder of this article is organised as follows. Section II presents the state of the art and reviews the selected solution approaches. Section III presents the mathematical model of the DBACP, including its solution representation, the objective function, the constraints and the student's dynamic progression.

The remainder of this paper is organized as follows. Section II presents the state of the art and reviews the selected solution approaches. Section III introduces the mathematical model of the DBACP, including its solution representation, objective function, constraints, and dynamic student progression.

II. State of the Art

One of the most relevant extensions of the original Balanced Academic Curriculum Problem (BACP) was proposed by Chiarandini et al. [2]. The authors introduced the Generalized Balanced Academic Curriculum Problem (GBACP), extending the classical formulation by incorporating multiple curricula, shared courses, and professors' preferences regarding teaching periods. They also showed that the traditional min–max objective, which minimizes the maximum credit load over all academic periods, may be insufficient to properly distinguish between solutions with the same maximum load. For this reason, they studied alternative balancing criteria based on the L_1, L_2, and L_∞ norms, selecting the quadratic L_2 criterion as the most suitable for their analysis.

To solve the GBACP, Chiarandini et al. combined Integer Programming with several local search approaches. Candidate solutions were represented as vectors in which each position corresponds to a course and its value indicates the assigned academic period. Two main neighborhood operators were considered: *MoveCourse*, which relocates one course to another feasible period, and *SwapCourses*, which exchanges the periods of two courses. These operators were combined with Simulated Annealing, Dynamic Tabu Search, and composite perturbation mechanisms called *kickers*. The experimental evaluation considered the three classical CSPLib instances and ten significantly larger real-world instances from the University of Udine, containing up to 307 courses and 37 curricula.

While the classical instances were solved easily, the generalized instances proved considerably harder.The experimental evaluation considered the three classical CSPLib instances and ten larger real-world instances from the University of Udine. While the classical instances were easily solved, the generalized instances were considerably more challenging. Local Search generally obtained better feasible solutions than Integer Programming, whereas IP was particularly useful for computing lower bounds used to assess solution quality. The main results are summarized in Table I.

TABLE I
RESULTS REPORTED BY CHIARANDINI ET AL.

Metric	Result
Evaluation	20 runs per instance with time limits of 320 and 3600 s.
Solution quality	Local Search obtained solutions approximately 9–60% above the available lower bounds.
Comparison	Local Search produced better upper bounds than Integer Programming in most instances.
Conclusion	IP was effective for obtaining lower bounds, while Local Search was more effective for finding good feasible solutions.

In 2013, Rubio et al. [3] proposed an Ant Colony Optimization approach based on the Best-Worst Ant System (BWAS) for the classical BACP. Their implementation represents a solution as a vector whose length is equal to the number of courses, where each position stores the academic period assigned to the corresponding course. The construction process is guided by a pheromone matrix and heuristic information that estimates the number of constraint violations produced by assigning a course to a particular period.

The proposed BWAS includes several mechanisms intended to balance intensification and diversification. All ants deposit pheromone according to the quality of their solutions, the best ant receives an additional reinforcement, and the worst solution is penalized through additional pheromone evaporation. A pheromone mutation mechanism introduces diversity, while a restart procedure is applied when the search stagnates. Additionally, a local search procedure attempts to move courses from the most loaded academic period to the least loaded one without introducing new constraint violations.

The method was evaluated on the three classical CSPLib instances and three real curricula using 50 independent executions per instance. BWAS reached the best-known values for all benchmark instances and matched or improved the original maximum load in the real curricula. Although some runs generated infeasible solutions, the best executions converged within the first 100 of the 1000 configured iterations. Table II summarizes the quantitative results.

Later, Rubio et al. [4] studied the application of the Firefly Algorithm (FA) to the BACP. In contrast to the previous vector representation, this work represents each curriculum as a binary matrix in which rows correspond to academic periods

TABLE II
RESULTS OBTAINED WITH BWAS.

Inst.	Best	Avg.	σ	Best (%)
BACP8	17	17.3	0.48	74%
BACP10	14	14.2	0.42	58%
BACP12	18	18.4	0.69	42%
INF	18	18.0	0.00	86%
ICI	18	18.0	0.00	86%
UPLA	13	14.0	0.66	20%

and columns to courses. A value of one indicates that a course is assigned to a specific period. Since the standard Firefly Algorithm operates in a continuous search space, the authors adapted it to the binary representation through a transfer function that transforms the continuous movement of each firefly into binary values.

Each firefly represents a candidate curriculum, and its brightness is associated with the quality of the solution. Less attractive fireflies move toward brighter ones according to the standard attraction mechanism of FA, while a random component contributes to the exploration of the search space. The algorithm uses parameters controlling attractiveness, light absorption, randomization, population size, and maximum number of generations.

The method was evaluated on three benchmark and three real instances using 50 independent executions. As shown in Table III, the Firefly Algorithm reached the best-known benchmark values and improved some real curricula, although the frequency of reaching the best solution varied considerably across instances.

TABLE III
RESULTS OBTAINED WITH THE FIREFLY ALGORITHM.

Inst.	Best	Avg.	Worst	Best (%)
BACP8	17	17.5	18	53%
BACP10	14	14.9	15	10%
BACP12	18	18.8	20	23%
REAL8	17	18.1	19	17%
REAL10	13	13.9	15	43%
REAL12	17	19.0	20	3%

More recently, Aguilera et al. [5] introduced a dynamic approach to academic curriculum balancing. Unlike the classical BACP, which constructs a curriculum assuming that students follow the predefined academic plan, the Dynamic Balanced Academic Curriculum Problem (DBACP) replans the remaining curriculum according to the current academic situation of each student. The approach considers already completed and pending courses, prerequisite relationships, academic load limits, and the semesters in which each course can be offered.

The proposed solution uses a vector representation in which each position corresponds to a course and its value represents the semester to which that course is assigned. Instead of minimizing only the maximum academic load, the authors evaluate a solution using the sum of squared deviations between the credit load of each semester and the average semester load.

This criterion provides additional information when comparing solutions with similar maximum loads.

A feasible initial solution is constructed using the concept of an assignable range for each course. Courses with smaller feasible ranges are prioritized, and each course is preferably assigned to the least-loaded semester within its valid range. After each assignment, the remaining ranges are updated to preserve prerequisite feasibility. The resulting solution is then improved using Tabu Search. The main movement consists of selecting courses from highly loaded semesters and relocating them to feasible semesters that produce the smallest increase in the evaluation function. A tabu list prevents recently performed movements from being immediately reversed, reducing the probability of cycling.

The approach was evaluated on the classical CSPLib benchmarks and real academic curricula from Universidad Técnica Federico Santa María. Tabu Search improved the load distribution in every reported instance while preserving the academic constraints. The reduction in both the squared load error and the difference between the most and least loaded semesters is shown in Table IV.

TABLE IV
RESULTS OBTAINED FOR THE DYNAMIC BACP.

Instance	Initial EAC	Final EAC	Max Credit Diff.
BACP8	7.875	1.875	$3 \rightarrow 1$
BACP10	22.400	2.400	$4 \rightarrow 1$
BACP12	18.000	0.000	$4 \rightarrow 0$
REAL10	12.900	2.900	$4 \rightarrow 2$
REAL7	22.857	6.857	$5 \rightarrow 3$

The experiments described in the papers were carried out under the following conditions, Chiarandini et al. used a 2.66 GHz dual quad-core server with 4 GB of RAM running Ubuntu Server, implementing their approach in C++ with EASYLOCAL++. Rubio et al. implemented the ACO approach in Java on a Windows PC with a 2.40 GHz Intel Core 2 Duo T8300, while their Firefly implementation used Java/NetBeans on a Windows 7 system with a 3.4 GHz AMD Phenom II X4 and 8 GB of RAM. Aguilera et al. used Python 3.12 on Windows 11 with an AMD Ryzen 7 5800H and 16 GB of DDR4 RAM.

III. MATHEMATICAL MODEL

The DBACP is formulated at a given academic period t, considering the current academic state of the student. The resulting curriculum plan includes only the courses that remain pending and is updated after the completion of each academic period.

A. Sets and Parameters

The following sets and parameters are considered:

- C: set of courses in the curriculum.
- $P = 1, \ldots, T$: set of academic periods.
- r_i: number of credits associated with course $i \in C$.
- $R \subseteq C \times C$: set of prerequisite relationships, where $(i, j) \in R$ indicates that course i is a prerequisite of course j.
- $A_t \subseteq C$: set of courses already passed before academic period t.
- U_t: set of courses still pending at period t.
- P_t: set of remaining academic periods.
- q_i: probability of successfully completing course i.
- $L_{\min}$ and $L_{\max}$: minimum and maximum credit load allowed in an academic period.
- $N_{\min}$ and $N_{\max}$: minimum and maximum number of courses allowed in an academic period.

The set of pending courses is defined as

$$U_t = C \setminus A_t, \tag{1}$$

and the remaining planning horizon is

$$P_t = t, t+1, \ldots, T. \tag{2}$$

Therefore, only courses belonging to U_t are considered when generating a new curriculum plan.

B. Decision Variables and Solution Representation

For each pending course $i \in U_t$, the decision variable x_i represents the academic period in which the course is assigned:

$$x_i \in P_t, \qquad \forall i \in U_t. \tag{3}$$

A candidate solution is represented by the vector

$$X_t = (x_i)_{i \in U_t}. \tag{4}$$

Each position of the vector corresponds to a pending course and its value indicates the academic period assigned to that course. Since each course has a single position in X_t, this representation implicitly guarantees that every pending course is assigned to exactly one academic period.

C. Academic Load

Let $Z(x_i = p)$ denote the indicator function

$$Z(x_i = p) = \begin{cases} 1, & \text{if } x_i = p, \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

The academic load of period p is then defined as

$$L_p(X_t) = \sum_{i \in U_t} r_i Z(x_i = p), \qquad \forall p \in P_t. \tag{6}$$

Similarly, the number of courses assigned to period p is

$$N_p(X_t) = \sum_{i \in U_t} Z(x_i = p), \qquad \forall p \in P_t. \tag{7}$$

D. Objective Function

The main objective is to minimize the maximum academic load among all remaining periods:

$$\min_{X_t} \left(\max_{p \in P_t} L_p(X_t) \right). \tag{8}$$

This objective seeks to avoid academic periods with excessively high credit loads and to distribute the remaining workload more evenly throughout the student's curriculum.

E. Constraints

1) Prerequisite Constraints: If course i is a prerequisite of course j and both courses are still pending, then course i must be assigned to an earlier academic period:

$$x_i < x_j, \qquad \forall (i,j) \in R \text{ such that } i,j \in U_t. \tag{9}$$

If the prerequisite course i already belongs to A_t, the prerequisite is considered satisfied and no additional ordering constraint is required.

2) Credit Load Constraints: The total number of credits assigned to each academic period must remain within the established limits:

$$L_{\min} \leq L_p(X_t) \leq L_{\max}, \qquad \forall p \in P_t. \tag{10}$$

3) Course Number Constraints: The number of courses assigned to each academic period must also remain within its minimum and maximum limits:

$$N_{\min} \leq N_p(X_t) \leq N_{\max}, \qquad \forall p \in P_t. \tag{11}$$

F. Secondary Balance Criterion

The min–max objective only considers the most heavily loaded academic period. Therefore, two solutions with the same maximum load may still present considerably different load distributions.

To distinguish between such solutions, the average credit load of the remaining curriculum is defined as

$$AVG_t = \frac{\sum_{i \in U_t} r_i}{|P_t|}. \tag{12}$$

A secondary balance measure is defined as

$$E(X_t) = \sum_{p \in P_t} \left(L_p(X_t) - AVG_t \right)^2. \tag{13}$$

This measure is not combined directly with the main objective. Instead, it is used as a secondary criterion when two candidate solutions have the same maximum academic load. In that case, the solution with the smaller value of $E(X_t)$ is preferred.

G. Dynamic Student Progress and Replanning

The dynamic component of the DBACP models the academic progress of the student after each period.

The set of courses actually taken during academic period t is determined by the current solution X_t as

$$K_t = \{\, i \in U_t : x_i = t \,\}. \tag{14}$$

For each course $i \in K_t$, let q_i denote the probability of successfully completing that course, where

$$0 \leq q_i \leq 1, \qquad \forall i \in C. \tag{15}$$

The academic outcome of each course taken during period t is modeled using a Bernoulli random variable:

$$Y_i^t \sim \text{Bernoulli}(q_i), \qquad \forall i \in K_t. \tag{16}$$

where

$$Y_i^t = \begin{cases} 1, & \text{if course } i \text{ is passed,} \\ 0, & \text{if course } i \text{ is failed.} \end{cases} \tag{17}$$

The set of courses successfully completed during period t is therefore

$$Pass_t = \{\, i \in K_t : Y_i^t = 1 \,\}. \tag{18}$$

Once the academic results of period t are obtained, the set of completed courses is updated as

$$A_{t+1} = A_t \cup Pass_t. \tag{19}$$

The new set of pending courses is then

$$U_{t+1} = C \setminus A_{t+1}. \tag{20}$$

Courses that were failed remain in U_{t+1} and are therefore considered again during the next replanning stage.

The remaining planning horizon is updated as

$$P_{t+1} = \{t+1, \ldots, T\}. \tag{21}$$

Using the updated sets A_{t+1}, U_{t+1}, and P_{t+1}, a new curriculum plan is generated:

$$X_{t+1} = (x_i)_{i \in U_{t+1}}. \tag{22}$$

Thus, the DBACP is modeled as a sequence of optimization problems. At each academic period, a curriculum plan is generated for the remaining courses, the student's academic results are simulated, and the problem is subsequently reformulated according to the courses successfully completed and those that remain pending.

REFERENCES

[1] C. Castro and S. Manzano, "Variable and value ordering when solving balanced academic curriculum problems," in *Proceedings of the 6th Workshop of the ERCIM WG on Constraints*, 2001.

[2] M. Chiarandini, L. D. Gaspero, S. Gualandi, and A. Schaerf, "The balanced academic curriculum problem revisited," *Journal of Heuristics*, vol. 18, no. 1, pp. 119–148, 2012.

[3] J.-M. Rubio, W. Palma, N. Rodriguez, R. Soto, B. Crawford, F. Paredes, and G. Cabrera, "Solving the balanced academic curriculum problem using the aco metaheuristic," *Mathematical Problems in Engineering*, vol. 2013, 2013.

[4] J. M. Rubio, R. Soto, H. Jorquera, J. Aguilera, and C. Vidal, "Solving the balanced academic curriculum problem using firefly algorithm," *Ingeniare. Revista chilena de ingenieria*, vol. 26, pp. 102–112, 2018.

[5] D. Aguilera, M. C. Riff, and P. Godoy, "Algorithm for the dynamic load balancing of university academic curricula," in *2025 44th International Conference of the Chilean Computer Science Society*, 2025.